Cross-Backend Compatibility of the Tensorial Risk Engine

T-Risk backend implementation study

April 23, 2026

Abstract

This report compares TensorFlow, PyTorch, and JAX implementations of the event-based tensorial risk formulation used in T-Risk. The canonical implementation is the TensorFlow backend in `tensor_engine.py`; PyTorch and JAX were implemented as independent ports with the same input arrays, outputs, and automatic-differentiation targets. Full-portfolio Bogotá tests show that the formulation is portable across tensor libraries: the forward loss matrices agree with TensorFlow at approximately float32 precision, and the gradients with respect to exposure, hazard, and event rates also agree closely. Vulnerability gradients exhibit larger relative L_2 differences because they aggregate many indexed contributions into a compact matrix, but their correlations remain above 0.999997. Performance differs strongly by backend and accelerator. TensorFlow Metal is the most balanced Apple backend for this workload, JAX Metal accelerates the joint backward pass but is experimental, and PyTorch MPS performs poorly for the indexing-heavy calculation tested here.

1 Purpose

The purpose of the cross-backend exercise is not to replace the TensorFlow engine, but to test whether the tensorial risk formulation implemented in TensorFlow has a broader computational representation. A successful port to PyTorch and JAX supports three claims:

- the formulation is expressed in standard tensor operations rather than TensorFlow-specific behavior;
- the automatic-differentiation targets are well defined across independent autodiff systems;
- the existing TensorFlow implementation is a defensible canonical reference for future backend and hardware comparisons.

All backends were tested with the same Bogotá inputs: `bogota_trisk_inputs.npz` and `bogota_hazard_chia.np` The four full-portfolio IM groups are shown in Table 1.

Table 1: Full Bogotá portfolio sizes.

IM group	Assets	Events	Typologies
`SA_0p1`	65,679	500	9
`SA_0p3`	64,658	500	16
`SA_0p6`	32,527	500	12
`SA_1p0`	43,674	500	48

2 Common Mathematical Kernel

All three implementations compute the same event-based loss matrix. For asset i and event q, the hazard intensity H_{iq} is mapped to a vulnerability curve interval on the common intensity grid. The mean damage ratio is computed by piecewise-linear interpolation of the typology-specific vulnerability curve C_{u_i}, and loss is

$$J_{iq} = v_i \, \mathrm{MDR}(u_i, H_{iq}).$$

Portfolio average annual loss is then

$$\mathrm{AAL} = \sum_i \sum_q \lambda_q J_{iq}.$$

The differentiable inputs are $\mathbf{v}$, $\mathbf{C}$, $\mathbf{H}$, and $\boldsymbol{\lambda}$. Typology indices and interpolation interval indices are integer-valued and non-differentiable. The OpenQuake-aligned clipping rule is preserved across the implementations: intensities above the vulnerability grid are clipped to the maximum grid point, while intensities below the first grid point produce zero damage.

3 Backends and Environments

TensorFlow is the canonical implementation and the numerical reference. It was tested on CPU and TensorFlow Metal GPU in `/Users/mac/virtual_environments/tensorial_engine_2`, using TensorFlow 2.16.2 and `tensorflow-metal` 1.2.0.

PyTorch was tested in the same environment using PyTorch 2.11.0 on CPU and MPS. The PyTorch backend is implemented in `tensor_engine_pytorch.py`.

JAX was tested in two configurations. Direct TensorFlow CPU comparison used JAX 0.4.28 and JAXLIB 0.4.28 in `tensorial_engine_2`. JAX Metal testing used a separate environment, `/Users/mac/virtual_environments/tensorial_engine_3`, with JAX 0.5.0, JAXLIB 0.5.0, `jax-metal` 0.1.1, and NumPy 2.4.4. The separate environment was necessary because current JAX Metal dependencies are not compatible with the TensorFlow 2.16.2 stack.

4 Numerical Compatibility

Figure 1 summarizes maximum relative L_2 errors across IM groups for the CPU ports relative to TensorFlow CPU. Both PyTorch CPU and JAX CPU reproduce the forward loss matrix at approximately 5×10^{-8} relative L_2 error. Exposure, hazard, and event-rate gradients also agree closely. The vulnerability-gradient relative error is larger for both ports, reaching approximately 5×10^{-3}, but the correlations remain above 0.999997.

Table 2: Maximum relative L_2 error across IM groups relative to TensorFlow CPU.

Quantity	PyTorch CPU	JAX CPU
Loss matrix	5.08e-08	5.19e-08
$\nabla_{\mathbf{C}}$	5.00e-03	5.17e-03
$\nabla_{\mathbf{v}}$	6.15e-08	6.52e-08
$\nabla_{\mathbf{H}}$	2.46e-07	3.40e-07
$\nabla_{\boldsymbol{\lambda}}$	7.40e-07	7.51e-07

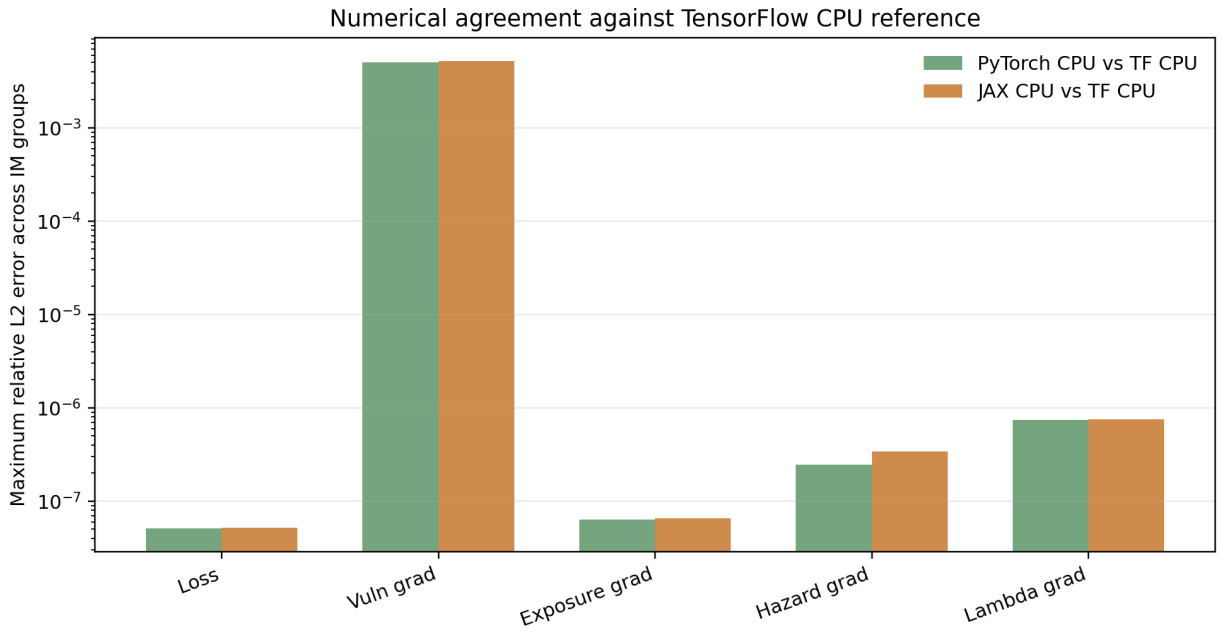

Figure 1: Maximum relative L_2 error across IM groups for PyTorch CPU and JAX CPU relative to the TensorFlow CPU reference.

The vulnerability-gradient behavior is expected for this computational graph. Many asset-event contributions are accumulated into the same vulnerability curve entries. Different backends use different accumulation order and kernel implementations; relative L_2 error can therefore be larger even when the field is structurally equivalent and highly correlated.

5 Forward Performance

Figure 2 compares forward-pass timings across the tested backends. TensorFlow Metal is consistently faster than TensorFlow CPU. PyTorch CPU is comparable to TensorFlow CPU, but PyTorch MPS is much slower. JAX CPU in the isolated JAX Metal environment is fast for the forward pass, while JAX Metal is slower than JAX CPU.

The common pattern is that the forward pass is not a dense linear-algebra kernel. It is dominated by clipping, `searchsorted`, flattened gather, reshaping, typology expansion, and reductions. This explains why accelerator performance is backend-dependent and why some GPU backends underperform CPU.

6 Backward Performance

Figure 3 compares the joint backward pass, which computes gradients of portfolio AAL with respect to $\mathbf{H}$, $\mathbf{C}$, $\mathbf{v}$, and $\boldsymbol{\lambda}$ in one reverse-mode pass.

TensorFlow Metal accelerates the backward pass by approximately 2.3–2.5× relative to TensorFlow CPU. JAX Metal accelerates the backward pass by approximately 5.2× relative to JAX CPU in the isolated JAX environment. PyTorch MPS does not provide useful acceleration for this implementation on the tested Apple hardware.

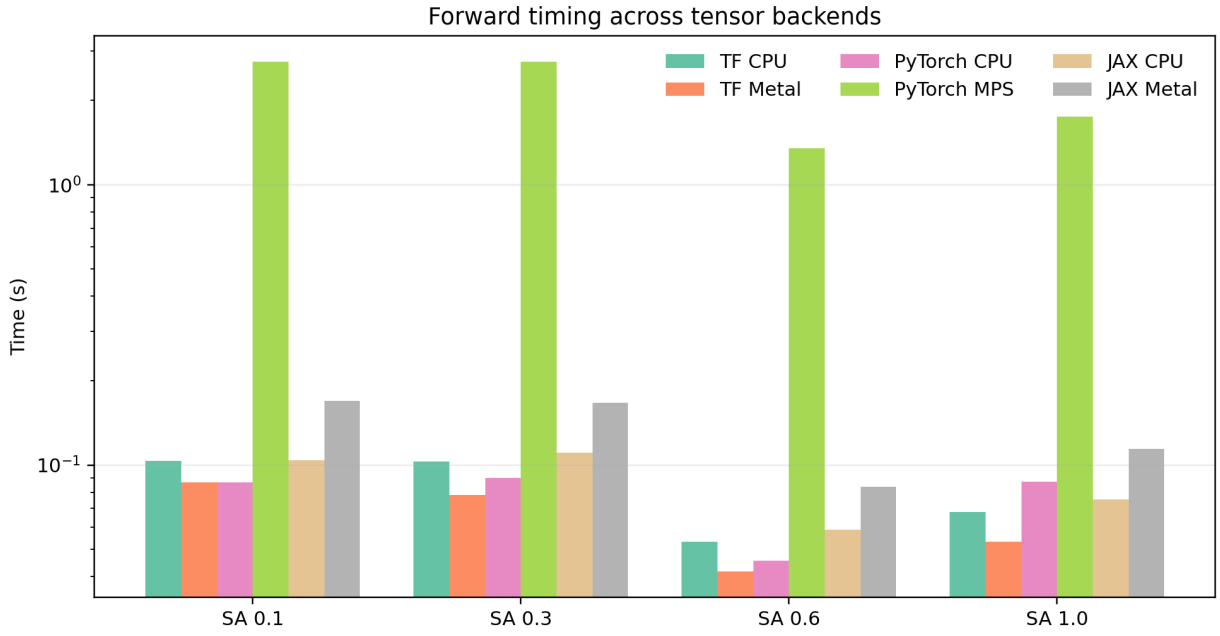

Figure 2: Forward timing across TensorFlow, PyTorch, and JAX backends. The y-axis is logarithmic because PyTorch MPS is much slower for this workload.

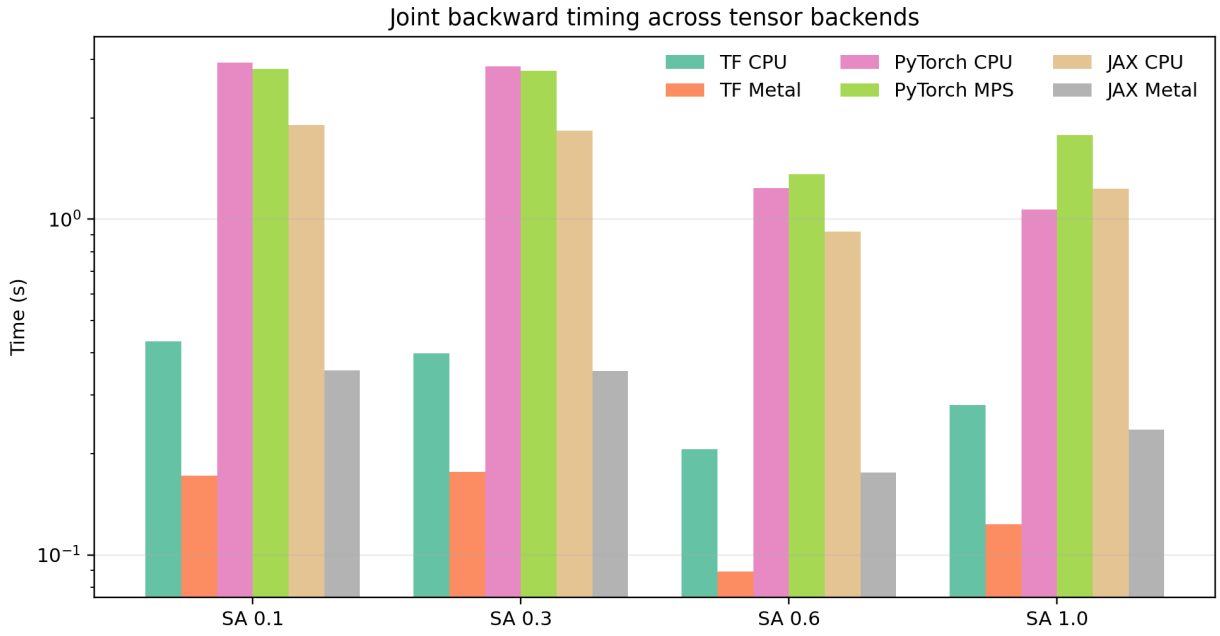

Figure 3: Joint backward timing across TensorFlow, PyTorch, and JAX backends. The y-axis is logarithmic.

Table 3: Representative full-portfolio mean timing ranges in seconds.

Backend/device	Forward range (s)	Backward range (s)
TensorFlow CPU	0.053–0.104	0.206–0.433
TensorFlow Metal	0.042–0.087	0.089–0.177
PyTorch CPU	0.046–0.090	1.071–2.936
PyTorch MPS	1.348–2.754	1.361–2.806
JAX CPU	0.059–0.110	0.920–1.909
JAX Metal	0.083–0.170	0.176–0.355

7 Interpretation

The three independent implementations support the portability of the tensorial risk formulation. The TensorFlow implementation is not relying on opaque or TensorFlow-specific semantics; it can be reproduced in PyTorch and JAX using standard tensor operations and reverse-mode autodiff.

Performance, however, is not portable by default. The workload is indexing-heavy and memory-sensitive. TensorFlow Metal handles the traced graph well on Apple hardware. PyTorch MPS appears poorly matched to the required gather and indexed-gradient accumulation pattern. JAX Metal is promising for the backward pass but remains experimental and required an additional post-compilation warm-up for stable hazard-gradient summaries.

The cleanest production conclusion is therefore conservative:

- TensorFlow remains the most mature and balanced backend for the current Apple M4 Pro setup.
- PyTorch is numerically compatible and should be tested on CUDA before drawing conclusions about PyTorch GPU performance.
- JAX is numerically compatible and attractive for compiled autodiff, but JAX Metal should be treated as exploratory until the backend matures.

8 Conclusions

The TensorFlow implementation is validated as a robust canonical reference for the tensorial risk formulation. Independent PyTorch and JAX ports reproduce the same loss and gradient structure on the full Bogotá portfolio. This supports the broader compatibility of the formulation and provides a defensible path for hardware-specific backend testing.

The next high-value benchmark should be a CUDA study comparing TensorFlow CUDA, PyTorch CUDA with and without `torch.compile`, and JAX CUDA steady-state JIT. CUDA is the appropriate platform for deciding whether PyTorch or JAX should become a production accelerator backend for larger T-Risk experiments.